

torch.nn.functional.pad#

<https://pytorch.org/docs/stable/generated/torch.nn.functional.pad.html>

Description:

The padding size by which to pad some dimensions of input are described starting from the last dimension and moving forward.

$$\lfloor \frac{\text{len}(\text{pad})}{2} \rfloor$$

dimensions of input will be padded. For example, to pad only the last dimension of the input tensor, then pad has the form

`(padding_left, padding_right)` (`\text{padding\_left}`, `\text{padding\_right}`)

`(padding_left, padding_right)`; to pad the last 2 dimensions of the input tensor, then use `(padding_left, padding_right, \text{padding\_left}`,

`\text{padding\_right}`, `(padding_left, padding_right`,

`padding_top, padding_bottom)` (`\text{padding\_top}`,

`\text{padding\_bottom}`) `padding_top, padding_bottom`); to pad the last 3

dimensions, use `(padding_left, padding_right, \text{padding\_left}`,

`\text{padding\_right}`, `(padding_left, padding_right`,

`padding_top, padding_bottom` `\text{padding\_top}`,

`\text{padding\_bottom}`) `padding_top, padding_bottom`

`padding_front, padding_back)` (`\text{padding\_front}`,

`\text{padding\_back}`) `padding_front, padding_back`).

See

`torch.nn.CircularPad2d`,

`torch.nn.ConstantPad2d`,

`torch.nn.ReflectionPad2d`, and `torch.nn.ReplicationPad2d` for concrete

examples on how each of the padding modes works. Constant pa

Function Signature

```
torch.nn.functional.pad(input, pad, mode='constant',  
value=None) → Tensor[source]#
```

Padding size:

Padding mode:

Parameters

Return type

Returns:

Tensor

Code Examples

```
>>> t4d = torch.empty(3, 3, 4, 2)
>>> p1d = (1, 1) # pad last dim by 1 on each side
>>> out = F.pad(t4d, p1d, "constant", 0) # effectively zero
padding
>>> print(out.size())
torch.Size([3, 3, 4, 4])
>>> p2d = (1, 1, 2, 2) # pad last dim by (1, 1) and 2nd to last
by (2, 2)
>>> out = F.pad(t4d, p2d, "constant", 0)
>>> print(out.size())
torch.Size([3, 3, 8, 4])
>>> t4d = torch.empty(3, 3, 4, 2)
>>> p3d = (0, 1, 2, 1, 3, 3) # pad by (0, 1), (2, 1), and (3, 3)
>>> out = F.pad(t4d, p3d, "constant", 0)
>>> print(out.size())
torch.Size([3, 9, 7, 3])
```

Notes & Warnings

NOTE

When using the CUDA backend, this operation may induce nondeterministic behaviour in its backward pass that is not easily switched off. Please see the notes on Reproducibility for background.

Detailed Documentation

Documentation

The padding size by which to pad some dimensions of input are described starting from the last dimension and moving forward. $\lfloor \frac{\text{len}(\text{pad})}{2} \rfloor$ dimensions of input will be padded. For example, to pad only the last dimension of the input tensor, then pad has the form `(padding_left, padding_right)`; to pad the last 2 dimensions of the input tensor, then use `(padding_left, padding_right, padding_top, padding_bottom)`; to pad the last 3 dimensions, use `(padding_left, padding_right, padding_top, padding_bottom, padding_front, padding_back)`.

See `torch.nn.CircularPad2d`, `torch.nn.ConstantPad2d`, `torch.nn.ReflectionPad2d`, and `torch.nn.ReplicationPad2d` for concrete examples on how each of the padding modes works. Constant padding is implemented for arbitrary dimensions. Circular, replicate and reflection padding are implemented for padding the last 3 dimensions of a 4D or 5D input tensor, the last 2 dimensions of a 3D or 4D input tensor, or the last dimension of a 2D or 3D input tensor.

When using the CUDA backend, this operation may induce nondeterministic behaviour in its backward pass that is not easily switched off. Please see the notes on Reproducibility for background.

input (Tensor) – N-dimensional tensor

pad (tuple) – m-elements tuple, where $2 \leq m \leq \text{input dimensions}$ and

mmm is even.

mode (str) – 'constant', 'reflect', 'replicate' or 'circular'. Default: 'constant'

value (Optional[float]) – fill value for 'constant' padding. Default: 0